MAPLY SERVICES

Informe Final

Maply Services: reportes colaborativos en tiempo real sobre lugares y servicios

Trabajo final de Prácticas Profesionalizantes — 6° 2° — Grupo 6
Escuela Técnica N°32 D.E. 14 “Gral. José de San Martín”

Figallo, Brunella — brunella.figalloet32@gmail.com
Kuo Lee, Felipe — felipekuolee.et32@gmail.com
Martinez, Lucca — lucca.martinez0@gmail.com

Participaron en la primera etapa: Llanos, Juan Pablo y Palacios, Valentina
Docentes: Prof. Damián Olaso y Prof. Gonzalo Consorti

Fecha de presentación: noviembre de 2026

Resumen

Maply Services es una aplicación móvil que brinda información en tiempo real sobre lugares y servicios a partir de reportes de la propia comunidad: filas, demoras, cierres, lugares llenos o cambios de recorrido del transporte. El objetivo es que cualquier persona pueda conocer el estado de un lugar antes de ir.

El trabajo recorrió todas las etapas de un proyecto de software: relevamiento con el cliente (entrevista y encuesta), análisis de requerimientos, diseño (casos de uso, DER, diagrama de clases, DFD y secuencia), planificación (Gantt, CPM, Planning Poker y burndown), desarrollo con Scrum en Jira, pruebas, despliegue en la nube y documentación.

Como resultado se entregó un sistema funcionando en producción: una app para Android (también compatible con iOS y web), una API REST documentada y una base de datos en la nube, con 325 tareas y 26 épicas completadas.

1. Introducción

La propuesta surge de una situación cotidiana que planteó el cliente: perder tiempo yendo a lugares de la ciudad sin poder informarse antes sobre los tiempos de espera o el estado del lugar. A través de la aplicación, los usuarios buscan un lugar (hospitales, bancos, comercios, transporte, oficinas públicas) y ven los reportes recientes de otras personas sobre situaciones como:

- Demoras en el servicio.
- Lugares llenos o con mucha fila.
- Cambios de recorrido del colectivo.
- Cierres temporales o falta de servicio.
- Estado general del lugar en un momento del día.

2. Estado del arte

Plataformas como Waze y Google Maps permiten compartir información en tiempo real: Waze se enfoca en el tránsito y Google Maps muestra reseñas, horarios y niveles de concurrencia estimados. Sin embargo, no están pensadas para situaciones puntuales del día a día, como la demora de un hospital, el estado momentáneo de un banco o un corte de recorrido, y su información no siempre está actualizada.

Maply Services se diferencia por enfocarse en reportes rápidos de la comunidad sobre tiempos de espera, concurrencia y estado momentáneo de los lugares, con una interfaz simple y un sistema de moderación para cuidar la calidad de la información.

3. Definición del problema y actores

El problema es la falta de información actualizada sobre el estado de lugares y servicios, que hace perder tiempo y genera viajes innecesarios. Los actores del sistema son:

- **Visitante:** consulta el mapa y los reportes sin crear una cuenta (modo invitado).

- **Usuario registrado:** publica y elimina sus reportes, denuncia reportes falsos y suma reputación (karma).
- **Administrador:** elimina cualquier reporte y bloquea usuarios que publican información falsa.

El análisis FODA del proyecto se encuentra en el repositorio (Diagrama FODA).

4. Solución propuesta

Una aplicación colaborativa donde los usuarios comparten en tiempo real el estado de lugares y servicios. Cada reporte tiene una categoría (mucha fila, lugar lleno, cerrado, demora, atención rápida, poco movimiento, cambio de recorrido u otro) y, para cada lugar, el sistema calcula su “estado actual” con los reportes de la última hora.

5. Análisis de requisitos

Los requerimientos funcionales y no funcionales, con su prioridad y su estado en la entrega final, están detallados en el documento “Funcionales y no funcionales – Maply Services” (versión 1.1). En resumen:

- Implementados: registro e inicio de sesión con recuperación de contraseña, búsqueda de lugares por nombre, categoría y cercanía, visualización y publicación de reportes, eliminación de reportes propios, denuncias, moderación por el administrador y mapa interactivo.
- Para la fase siguiente: favoritos, notificaciones por email, edición de reportes, panel de estadísticas y filtros avanzados.

6. Especificaciones del sistema

6.1 Arquitectura

El sistema usa una arquitectura cliente-servidor en tres capas. La app se comunica solo con la API por HTTPS, y la API es la única que accede a la base de datos.

6.2 Tecnologías utilizadas

Capa	Tecnología
Frontend (app)	React Native + Expo, Expo Router, TypeScript, NativeWind, Mapbox, i18next
Backend (API REST)	Node.js + Express, JWT, bcryptjs, express-validator, helmet, rate limiting
Base de datos	MySQL (XAMPP en desarrollo; Aiven en producción)
Hosting	Render (API) y Aiven (base de datos), en planes gratuitos
Documentación de la API	OpenAPI 3.0 (Swagger), en la ruta /api-docs
Calidad y despliegue	Tests con node:test, integración continua con GitHub Actions y APK con EAS Build

6.3 Justificación de las tecnologías

- **React Native con Expo:** un mismo código para Android, iOS y web, con acceso a GPS y almacenamiento seguro sin tocar código nativo.
- **Node.js con Express:** usa el mismo lenguaje que el frontend (JavaScript/TypeScript), tiene un ecosistema amplio para APIs REST y permitió reutilizar el trabajo ya avanzado. En la versión inicial de este informe se había evaluado Java; la decisión final quedó documentada en la investigación de frameworks de backend.
- **MySQL:** base de datos relacional confiable y conocida, adecuada para la información estructurada del sistema.
- **Mapbox:** plan gratuito amplio y estilos de mapa personalizables, según la investigación de APIs de mapas.

6.4 Requerimientos de hardware y software

- Desarrollo: procesador i3 / Ryzen 3 o superior, 8 GB de RAM, Windows 10/11, Node.js, Visual Studio Code, XAMPP y Android Studio.
- Uso de la app: teléfono Android 8 o superior (o iPhone actualizado) con conexión a internet.

6.5 Módulos del sistema

Módulo	Funciones
Autenticación	Registro, inicio y cierre de sesión, recuperación de contraseña, eliminación de cuenta
Lugares	Búsqueda por nombre, categoría y cercanía; mapa; propuesta de lugares nuevos
Reportes	Publicación, listado, estado actual del lugar, traducción y eliminación
Moderación	Denuncias, ocultamiento automático, eliminación por el administrador y bloqueo de usuarios
Usuario	Perfil, “miembro desde”, idioma, tema claro/oscuro y karma

7. Análisis del sistema

Los diagramas actualizados están en el repositorio del proyecto: casos de uso, diagrama de clases (11 clases), DER, DFD y diagrama de secuencia. La base de datos tiene 10 tablas: usuario, rol, lugar, reporte, denuncia, favorito, notificación, tokens_revocados, recuperacion_contrasena y traduccion_reporte, además de un trigger que suma karma por cada reporte.

8. Planificación y gestión del proyecto

El proyecto se gestionó con Scrum en Jira: 325 tareas atómicas agrupadas en 26 épicas, en sprints de dos semanas, con estimaciones por Planning Poker y seguimiento de ceremonias en AgileBox. La planificación se reflejó en el diagrama de Gantt (ProjectLibre) y en el CPM;

la fase de desarrollo (julio a noviembre) tiene un camino crítico de 55 días hábiles. El avance se midió con un burndown chart.

Las etapas del proyecto fueron: relevamiento, análisis del problema, definición de requisitos, diseño del sistema y de la base de datos, desarrollo, pruebas, corrección de errores, despliegue y documentación.

9. Resultados obtenidos

9.1 Implementación

Se desarrolló el sistema completo por etapas y quedó funcionando en producción:

- App para Android generada con EAS Build, también compatible con iOS y web.
- API REST con 16 endpoints documentados en Swagger: https://maply-backend-3sxm.onrender.com/api-docs.
- Base de datos MySQL en la nube con conexión cifrada.
- Despliegue automático: cada cambio en la rama main se prueba y se publica solo.

9.2 Pruebas realizadas

Prueba	Resultado
Registro y login (datos válidos e inválidos)	Correcto: 201/200 y 400/401 según corresponda (Postman y tests)
Publicación de reportes y límite de uno por hora por lugar	Correcto
Búsqueda de lugares y orden por cercanía	Correcto
Estado actual de un lugar	Correcto
Denuncias y ocultamiento automático con 3 denuncias	Correcto
Eliminación de reportes (autor, otro usuario, administrador)	Correcto: el autor y el admin eliminan; otro usuario recibe 403
Cierre de sesión y reuso de un token revocado	Correcto: la API responde 401
Tests automáticos del backend	24 de 24 pasan en GitHub Actions

9.3 Resultados

Se obtuvo una aplicación funcional, clara y fácil de usar, que cumple las tareas principales definidas al inicio: navegación simple, registro e ingreso, publicación y consulta de reportes, organización de la información por lugar y moderación del contenido.

10. Conclusiones

El proyecto nos permitió aplicar lo aprendido en nuestra formación técnica en todas las etapas de un sistema real: análisis, planificación, diseño, desarrollo, pruebas, despliegue y documentación.

Comprobamos la importancia de relevar las necesidades reales antes de programar y de mantener la documentación al día: cuando el código y los documentos se contradecían, perdíamos tiempo. También aprendimos a medir el avance con el burndown para corregir a tiempo y a redistribuir el trabajo cuando el equipo pasó de cinco a tres integrantes.

Consideramos que Maply Services es una solución viable y útil para una necesidad real de la vida cotidiana, con una base sólida para crecer: favoritos, notificaciones, panel de estadísticas y la expansión a otras ciudades.